DEVOPS INTERVIEW PREPARATION GUIDE

Top 20 Questions & Answers

CI/CD | Docker | Kubernetes | AWS | Ansible | Monitoring | Linux

HOW TO USE THIS GUIDE

This guide covers the 20 most frequently asked DevOps interview questions. **Read each answer carefully, understand the concept, and try to relate it to a real-world scenario. Practice the code snippets in your terminal.**

SECTION 1: DEVOPS FUNDAMENTALS

Q1. What is DevOps?

Definition

DevOps is a culture and set of practices that unifies software Development (Dev) and IT Operations (Ops). The goal is to shorten the software development lifecycle while delivering high-quality software continuously.

Simple Explanation

- **Traditional approach:** Developers write code. Operations team deploys it. These two teams often worked in silos, leading to delays and blame games.
- **DevOps approach:** Both teams collaborate throughout the lifecycle - from planning to deployment to monitoring - using automation and shared tools.

Key Principles

- **Collaboration:** Dev and Ops work together.
- **Automation:** Automate repetitive tasks (builds, tests, deployments).
- **Continuous Improvement:** Constantly measure, learn, and improve.
- **Fast Feedback:** Catch bugs early with automated testing and monitoring.

Real-World Example

Netflix deploys code thousands of times per day using DevOps practices. If a bug is found, it is detected automatically by monitoring and fixed within minutes - without downtime.

Q2. What are the Benefits of DevOps?

- **Faster Delivery:** Release software in hours or days, not weeks or months.
- **Improved Collaboration:** Dev and Ops teams share responsibility and communicate better.

- **Higher Quality:** Automated testing catches bugs before they reach production.
- **Reduced Failure Rate:** Smaller, incremental deployments reduce the risk of large failures.
- **Faster Recovery:** When something breaks, the mean time to recover (MTTR) is much lower.
- **Scalability:** Infrastructure can be scaled automatically using IaC tools like Terraform or Ansible.

Real-World Example

Amazon went from deploying software once every few months to deploying every 11.7 seconds after adopting DevOps. This means faster feature releases and quicker bug fixes.

Q3. What is a CI/CD Pipeline?

Definition

A CI/CD pipeline is an automated workflow that takes code from a developer's commit all the way to production. CI stands for Continuous Integration and CD stands for Continuous Delivery or Continuous Deployment.

Stages of a Typical Pipeline

- **Source:** Developer commits code to a Git repository (GitHub, GitLab, Bitbucket).
- **Build:** Code is compiled and dependencies are resolved.
- **Test:** Automated unit tests, integration tests, and security scans run.
- **Package:** Code is packaged into a Docker image or a JAR/WAR file.
- **Deploy:** Package is deployed to a staging or production environment.
- **Monitor:** Application is monitored for errors and performance issues.

Real-World Example

A developer pushes a bug fix to GitHub. Jenkins automatically picks it up, builds the project, runs 500 unit tests, builds a Docker image, and deploys it to production - all within 10 minutes, with zero manual steps.

Simple Pipeline Code (Jenkinsfile)

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps { sh 'mvn clean package' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Deploy') {
      steps { sh './deploy.sh' }
    }
  }
}
```

Q4. What is the Difference Between Continuous Delivery and Continuous Deployment?

- **Continuous Delivery:** Code is automatically tested and prepared for release. A human must manually approve and trigger the deployment to production.
- **Continuous Deployment:** Every code change that passes automated tests is deployed to production automatically - no human approval required.

Analogy

Continuous Delivery is like a car assembled on a production line and parked in the showroom - ready to be driven out, but a salesperson (human) must hand over the keys. Continuous Deployment means the car drives itself out of the factory and is ready for customers immediately.

When to Use Which

- **Continuous Delivery:** When releases require compliance sign-off, business validation, or regulatory approval.
- **Continuous Deployment:** When you have very high test confidence and want maximum delivery speed (e.g., SaaS products).

SECTION 2: GIT

Q5. What is the Difference Between Git and GitHub?

- **Git:** A distributed version control system (VCS) installed on your local machine. It tracks changes to files and allows multiple people to collaborate on the same codebase.
- **GitHub:** A cloud-based hosting platform for Git repositories. It adds collaboration features like Pull Requests, Issues, Actions (CI/CD), and team access controls.

Analogy

Git is like Microsoft Word's track changes feature on your computer. GitHub is like Google Docs - it hosts the document in the cloud and lets multiple people collaborate with version history.

Common Git Commands

```
git init                        # Initialize a new repository
git clone <url>                 # Copy a remote repository locally
git add .                       # Stage all changes
git commit -m 'fix: bug'        # Commit with a message
git push origin main            # Push to remote
git pull origin main            # Pull latest changes
git status                      # Check current state
git log --oneline               # View commit history
```

Q6. What is a Branching Strategy in Git?

Definition

A branching strategy defines how a team creates, names, and merges branches in a Git repository to organize work and prevent conflicts.

Popular Strategies

- **Git Flow:** Uses main, develop, feature/*, release/*, and hotfix/* branches. Suitable for projects with scheduled releases.
- **GitHub Flow:** Uses only main and feature branches. Simple and fast. Suitable for continuous deployment.
- **Trunk-Based Development:** All developers commit directly to main (trunk) using short-lived feature branches. Used at Google and Facebook.

Typical Git Flow Example

```
git checkout -b feature/login-page    # Create feature branch
# ... make changes ...
git push origin feature/login-page    # Push feature branch
# Open Pull Request -> Review -> Merge into develop
# After testing: merge develop into main -> Release
```

Q7. What is the Difference Between Git Merge and Git Rebase?

- **Git Merge:** Combines two branches by creating a new merge commit. Preserves full history. Safe to use on shared branches.
- **Git Rebase:** Moves or re-applies commits from one branch on top of another. Creates a linear, cleaner history. Avoid on shared branches as it rewrites history.

Analogy

Merge is like stapling two documents together - you can see they were separate. Rebase is like rewriting document B as if it was always written after document A - the history looks cleaner but the originals are rewritten.

```
# Merge
git checkout main
git merge feature/login-page   # Creates a merge commit

# Rebase
git checkout feature/login-page
git rebase main               # Re-applies feature commits on top of main
```

Key Rule

- **Never rebase** a public/shared branch. Only rebase your local feature branches before merging.

SECTION 3: CI/CD TOOLS

Q8. What is Jenkins?

Definition

Jenkins is an open-source automation server written in Java. It is the most widely used CI/CD tool. Jenkins automates the building, testing, and deploying of applications.

Key Features

- **Pipeline as Code:** Define build steps in a Jenkinsfile stored in your repository.
- **Plugin Ecosystem:** Over 1800 plugins for Git, Docker, Kubernetes, AWS, Slack, etc.

- **Distributed Builds:** Use master-agent architecture to run builds in parallel across multiple machines.
- **Open Source:** Free to use and widely supported by the community.

Real-World Example

A team stores a Jenkinsfile in their GitHub repo. Every time a developer pushes code, GitHub notifies Jenkins via a webhook. Jenkins pulls the code, runs Maven build, executes tests, builds a Docker image, and pushes it to Docker Hub - all automatically.

Q9. What is a Jenkins Pipeline?

Definition

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins. It is defined in a Jenkinsfile using Groovy-based DSL.

Two Types

- **Declarative Pipeline:** Simpler, structured syntax. Recommended for most use cases.
- **Scripted Pipeline:** More flexible Groovy scripting. Used for complex logic.

Declarative Jenkinsfile Example

```
pipeline {
  agent any
  environment {
    IMAGE_NAME = 'myapp:latest'
  }
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/org/myapp.git' }
    }
    stage('Build Docker Image') {
      steps { sh 'docker build -t $IMAGE_NAME .' }
    }
    stage('Push to Registry') {
      steps {
        sh 'docker push myregistry/$IMAGE_NAME'
      }
    }
    stage('Deploy') {
      steps { sh 'kubectl apply -f deployment.yaml' }
    }
  }
  post {
    failure { mail to: 'team@company.com', subject: 'Build Failed' }
  }
}
```

Q10. What is the Difference Between Build and Deploy?

- **Build:** The process of compiling source code, resolving dependencies, running unit tests, and packaging the application into an artifact (JAR, WAR, Docker image, binary).

- **Deploy:** The process of taking that built artifact and placing it into a target environment (development, staging, or production) so that users can access it.

Analogy

Build is like baking a cake (raw ingredients combined into a finished product). Deploy is like delivering that cake to the customer's table.

- **Build tools:** Maven, Gradle, npm, pip, Make.
- **Deploy tools:** Ansible, Kubernetes, Helm, Capistrano, AWS CodeDeploy.

SECTION 4: CONTAINERS & DOCKER

Q11. What is Docker and Why is it Used?

Definition

Docker is an open-source containerization platform that allows developers to package an application and all its dependencies into a lightweight, portable unit called a container.

Why Use Docker

- **Works Everywhere:** Eliminates the "it works on my machine" problem. A Docker container runs the same on a developer laptop, staging server, or AWS cloud.
- **Isolation:** Each container is isolated from others. No dependency conflicts between applications.
- **Lightweight:** Containers share the host OS kernel, making them much faster and lighter than Virtual Machines.
- **Scalability:** Easily scale containers up or down based on demand using tools like Kubernetes.

Real-World Example

A Python app that requires Python 3.11, specific libraries, and environment variables is packaged into a Docker image. Any team member or cloud server can run it with a single command without installing anything manually.

Q12. What is the Difference Between a Docker Image and a Container?

- **Docker Image:** A read-only blueprint or template used to create containers. It contains the application code, runtime, libraries, and configuration. Think of it as a class in OOP.
- **Docker Container:** A running instance of a Docker image. Multiple containers can be created from the same image. Think of it as an object (instance) created from a class.

Analogy

A Docker Image is like a cookie cutter. A Docker Container is the actual cookie cut from it. You can use the same cutter to make thousands of identical cookies.

```
# Pull an image from Docker Hub
docker pull nginx:latest

# Run a container from the image
docker run -d -p 8080:80 --name my-nginx nginx:latest
```

```
# List running containers
docker ps

# Stop and remove the container
docker stop my-nginx && docker rm my-nginx
```

Q13. What is a Dockerfile? Write a Sample Example.

Definition

A Dockerfile is a text file containing a set of instructions that Docker uses to automatically build a Docker image. Each instruction creates a new layer in the image.

Common Dockerfile Instructions

- **FROM:** Base image to start from.
- **WORKDIR:** Set the working directory inside the container.
- **COPY / ADD:** Copy files from host to container.
- **RUN:** Execute a command during image build.
- **EXPOSE:** Document which port the app runs on.
- **CMD / ENTRYPOINT:** Command to run when the container starts.

Sample Dockerfile - Node.js Application

```
# Step 1: Use official Node.js base image
FROM node:18-alpine

# Step 2: Set working directory
WORKDIR /app

# Step 3: Copy package files and install dependencies
COPY package*.json ./
RUN npm install --production

# Step 4: Copy application source code
COPY . .

# Step 5: Expose the port
EXPOSE 3000

# Step 6: Start the application
CMD ["node", "server.js"]
# Build the image
docker build -t my-node-app:1.0 .

# Run a container
docker run -d -p 3000:3000 my-node-app:1.0
```

SECTION 5: ORCHESTRATION - KUBERNETES

Q14. What is Kubernetes? Explain Pods, Nodes, and Cluster.

Definition

Kubernetes (K8s) is an open-source container orchestration platform developed by Google. It automates deployment, scaling, and management of containerized applications.

Core Concepts

- **Cluster:** The entire Kubernetes environment. It consists of a Control Plane (master) and multiple Worker Nodes.
- **Node:** A physical or virtual machine in the cluster that runs containerized workloads. Each node has a Kubelet agent, container runtime (Docker/containerd), and Kube-proxy.
- **Pod:** The smallest deployable unit in Kubernetes. A Pod wraps one or more containers that share the same network namespace and storage. Pods are ephemeral - they can be restarted or rescheduled on different nodes.
- **Deployment:** A higher-level abstraction that manages Pods. It ensures the desired number of Pods are always running and handles rolling updates and rollbacks.
- **Service:** A stable network endpoint that exposes Pods. Since Pod IPs change when restarted, Services provide a consistent DNS name and IP.

Real-World Example

An e-commerce app runs 10 replicas of its order-processing service on a Kubernetes cluster. When traffic spikes on sale day, Kubernetes Horizontal Pod Autoscaler automatically increases replicas to 50. When a Pod crashes, Kubernetes restarts it automatically within seconds.

Basic kubectl Commands

```
kubectl get pods                    # List all pods
kubectl get nodes                   # List all nodes
kubectl apply -f deployment.yaml    # Apply a configuration
kubectl describe pod <pod-name>     # View pod details
kubectl logs <pod-name>             # View container logs
kubectl delete pod <pod-name>       # Delete a pod
```

SECTION 6: CLOUD COMPUTING - AWS

Q15. Explain AWS Core Services: EC2, S3, IAM, and VPC

EC2 - Elastic Compute Cloud

- EC2 provides resizable virtual servers (instances) in the cloud.
- **Use case:** Hosting web servers, running batch jobs, running databases.
- You choose the instance type (CPU, RAM), OS (Amazon Linux, Ubuntu, Windows), and storage. You pay per hour/second of usage.

S3 - Simple Storage Service

- S3 is a highly durable and scalable object storage service. It stores files (objects) in buckets.
- **Use case:** Storing application logs, static website assets, database backups, Docker image layers, and data lake storage.
- Durability: 99.999999999% (11 nines). Objects can be up to 5 TB in size.

IAM - Identity and Access Management

- IAM controls who can access AWS services and what actions they can perform. It manages Users, Groups, Roles, and Policies.
- **Best Practice:** Follow Principle of Least Privilege - grant only the minimum permissions required.
- **Use case:** Allow a Jenkins server (EC2 instance) to push Docker images to ECR using an IAM Role - no credentials stored on the server.

VPC - Virtual Private Cloud

- VPC is a logically isolated section of the AWS cloud where you can launch AWS resources in a private network that you define.
- You control IP address ranges (CIDR blocks), subnets (public and private), route tables, internet gateways, and security groups.
- **Use case:** Put your database in a private subnet (no internet access). Put your web server in a public subnet. Control traffic with Security Groups and NACLs.

SECTION 7: AUTOMATION - ANSIBLE & IAC

Q16. What is Ansible and What is Infrastructure as Code (IaC)?

What is Ansible

Ansible is an open-source IT automation tool that automates configuration management, application deployment, and task automation. It uses YAML-based playbooks and is agentless - it only needs SSH access to target machines.

- **Agentless:** No software needs to be installed on managed servers.
- **Idempotent:** Running the same playbook multiple times produces the same result.
- **Human-Readable:** YAML syntax is easy to read and write.

Sample Ansible Playbook

```
---
- name: Install and start Nginx
  hosts: webservers
  become: yes          # Run as root
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present

    - name: Start Nginx service
      service:
        name: nginx
        state: started
        enabled: yes
```

What is Infrastructure as Code (IaC)

IaC is the practice of managing and provisioning infrastructure (servers, networks, databases) through machine-readable configuration files rather than manual processes.

- **Benefits:** Version-controlled infrastructure, repeatable environments, faster provisioning, reduced human error.
- **Tools:** Terraform (cloud-agnostic), AWS CloudFormation (AWS-specific), Pulumi, Ansible.
- **Example:** Instead of clicking through AWS Console to create 10 EC2 instances, you write a Terraform script that creates them in 2 minutes.

SECTION 8: MONITORING & LOGGING

Q17. What is Prometheus, Grafana, and the ELK Stack?

Prometheus

- **What it is:** An open-source monitoring system that collects and stores time-series metrics (CPU usage, memory, request count, error rates) by scraping HTTP endpoints.
- **How it works:** Applications expose a /metrics endpoint. Prometheus scrapes this endpoint at regular intervals and stores the data in its time-series database.
- **Use case:** Monitor a Kubernetes cluster - track how many requests per second each Pod is handling.

Grafana

- **What it is:** An open-source visualization and dashboarding tool. It connects to data sources like Prometheus, Elasticsearch, and CloudWatch to display beautiful real-time dashboards.
- **Use case:** Create a dashboard that shows CPU usage, memory consumption, and API response times - all in one view. Set up alerts to notify Slack when CPU exceeds 80%.

ELK Stack - Elasticsearch, Logstash, Kibana

- **Elasticsearch:** A distributed search and analytics engine. It stores and indexes log data so it can be searched quickly.
- **Logstash:** A data processing pipeline that ingests logs from multiple sources (files, syslog, databases), transforms them, and sends them to Elasticsearch.
- **Kibana:** A visualization layer on top of Elasticsearch. It lets you search logs, create visualizations, and build dashboards.
- **Use case:** All microservices send logs to Logstash. Logstash parses and sends them to Elasticsearch. On-call engineers search Kibana when a production issue occurs to find the root cause in seconds.

SECTION 9: LINUX FOR DEVOPS

Q18. Most Important Linux Commands for DevOps

File and Directory Operations

```
ls -la                # List files with details and hidden files
cd /var/log           # Change directory
pwd                   # Print working directory
mkdir -p /app/data    # Create directories recursively
cp -r src/ dest/      # Copy directory recursively
mv old.txt new.txt    # Move or rename file
rm -rf /tmp/cache     # Remove directory forcefully
find / -name '*.log' # Find all .log files
```

Process and System Monitoring

```
top                   # Real-time CPU and memory usage
htop                  # Interactive process viewer
ps aux                # List all running processes
kill -9 <PID>         # Force kill a process by PID
df -h                 # Disk space usage (human readable)
du -sh /var/log       # Size of a specific directory
free -m               # Memory usage in MB
uptime                # How long the system has been running
```

Network Commands

```
curl -I https://example.com    # Check HTTP response headers
wget https://example.com/file  # Download a file
netstat -tuln                  # List open ports
ss -tuln                       # Modern alternative to netstat
ping 8.8.8.8                   # Test network connectivity
nslookup example.com           # DNS lookup
traceroute example.com         # Trace network path
```

Log Viewing and File Inspection

```
tail -f /var/log/nginx/error.log   # Follow log in real-time
grep 'ERROR' app.log               # Search for ERROR in file
grep -r 'OutOfMemory' /var/log/    # Recursive search
awk '{print $1}' access.log        # Print first column
cat /etc/os-release                # Check OS version
journalctl -u nginx -n 100         # View last 100 lines of service log
```

Permission and User Management

```
chmod 755 script.sh       # Set permissions (rwxr-xr-x)
chown user:group file.txt # Change file owner
sudo su -                 # Switch to root user
adduser devops            # Add new user
usermod -aG sudo devops   # Add user to sudo group
```

SECTION 10: REAL-TIME SCENARIOS

Q19. How Do You Handle a Deployment Failure and Rollback?

Scenario

A new version of your application was deployed but users are reporting errors. The on-call alert fires. What do you do?

Step-by-Step Response

- **Step 1 - Detect:** Monitoring alerts (Prometheus/Grafana) fire. Check error rate and response time dashboards.
- **Step 2 - Communicate:** Notify the team in Slack. Open an incident channel. Assign incident commander.
- **Step 3 - Assess:** Check deployment logs in Jenkins or your CD tool. Look at application logs in Kibana or CloudWatch.
- **Step 4 - Rollback:** If the new version is the cause, immediately rollback to the last known-good version.
- **Step 5 - Verify:** Confirm error rates drop back to normal after rollback. Notify stakeholders.
- **Step 6 - Post-Mortem:** After the incident, do a blameless post-mortem. Document root cause, timeline, and prevention actions.

Rollback Commands

```
# Kubernetes rollback
kubectl rollout undo deployment/my-app

# View rollout history
kubectl rollout history deployment/my-app

# Rollback to specific revision
kubectl rollout undo deployment/my-app --to-revision=3

# Jenkins - trigger previous successful build's deploy job
```

Q20. How Do You Debug a Production Issue?

Scenario

Your application in production is slow or returning 500 errors. Users are complaining. You have never seen this error before. Walk through your debugging process.

Systematic Debugging Approach

- **1. Check Monitoring Dashboards:** Look at Grafana/Prometheus - is it CPU, memory, or disk? Is it all services or one specific service?
- **2. Check Application Logs:** Search Kibana or use kubectl logs / journalctl for ERROR and WARN messages. Look for stack traces.
- **3. Check Recent Changes:** Was there a deployment in the last few hours? A config change? A database migration? Check the deployment pipeline.
- **4. Check Infrastructure:** Is the database reachable? Are external APIs timing out? Use ping, curl, and netstat to test connectivity.
- **5. Check Resource Limits:** Is a pod hitting its memory limit and getting OOMKilled? Is disk 100% full on the server?
- **6. Reproduce the Issue:** Try to replicate the error in a staging environment with the same inputs.

- **7. Fix and Deploy:** Apply the fix, test in staging, deploy to production, monitor for 15-30 minutes.

Useful Debugging Commands

```
# Check pod status and events
kubectl describe pod <pod-name>

# View live logs
kubectl logs -f <pod-name> --previous

# Check resource usage
kubectl top pods
kubectl top nodes

# Check disk space
df -h

# Check which process is using most memory
ps aux --sort=-%mem | head -10

# Test API endpoint
curl -v https://api.myapp.com/health
```

QUICK REVISION CHEAT SHEET

1-Page Summary - Read Before Your Interview

DevOps Fundamentals

- **DevOps:** Culture of Dev + Ops collaboration. Automate everything.
- **CI:** Continuous Integration - merge code often, build and test automatically.
- **CD (Delivery):** Code ready to deploy; human approves. CD (Deployment): Auto-deploy on every passing test.
- **Benefits:** Faster releases, fewer bugs, faster recovery, better collaboration.

Git

- **Git:** Local VCS. GitHub: Cloud hosting for Git repos.
- **Merge:** Combines branches, creates merge commit, preserves history.
- **Rebase:** Re-applies commits on top of another branch, creates clean linear history.
- **Git Flow:** main > develop > feature/* > release/* > hotfix/*

Jenkins & CI/CD

- **Jenkins:** Open-source automation server for CI/CD. Uses Jenkinsfile (Groovy DSL).
- **Pipeline Stages:** Checkout > Build > Test > Package > Deploy > Monitor
- **Build:** Compile + test + package. Deploy: Place artifact in target environment.

Docker & Containers

- **Docker:** Containerization platform. Solves 'works on my machine' problem.
- **Image:** Read-only blueprint (like a class). Container: Running instance (like an object).
- **Dockerfile:** FROM > WORKDIR > COPY > RUN > EXPOSE > CMD
- Key commands: docker build, docker run, docker push, docker ps, docker logs

Kubernetes

- **Kubernetes:** Container orchestration. Automates deployment, scaling, healing.
- **Cluster:** Master (Control Plane) + Worker Nodes.
- **Pod:** Smallest unit. Wraps one or more containers.
- **Service:** Stable network endpoint for Pods. Deployment: Manages Pod replicas and updates.
- Key commands: kubectl get pods/nodes, kubectl apply, kubectl logs, kubectl rollout undo

AWS

- **EC2:** Virtual servers in the cloud.

- **S3:** Object storage. Durable, scalable, cheap.
- **IAM:** Access control. Users, Roles, Policies. Principle of Least Privilege.
- **VPC:** Isolated private network. Public subnet (internet-facing). Private subnet (internal).

Ansible & IaC

- **Ansible:** Agentless automation via SSH. YAML Playbooks. Idempotent.
- **IaC:** Manage infrastructure via code (Terraform, CloudFormation, Ansible).

Monitoring & Logging

- **Prometheus:** Scrapes and stores metrics. PromQL for querying.
- **Grafana:** Visualizes metrics from Prometheus. Dashboards + Alerts.
- **ELK Stack:** Elasticsearch (store) + Logstash (ingest) + Kibana (visualize). For log management.

Linux Essentials

- File: ls, cd, cp, mv, rm, find, chmod, chown
- Process: top, ps aux, kill, df -h, free -m
- Network: curl, ping, netstat, ss, nslookup
- Logs: tail -f, grep, awk, journalctl

Incident Response

- **Deployment Failure:** Detect > Communicate > Assess > Rollback > Verify > Post-Mortem
- **Production Debug:** Check dashboards > logs > recent changes > infra > resources > reproduce > fix
- **Kubernetes Rollback:** kubectl rollout undo deployment/<name>

Best of luck with your DevOps interview! Remember: interviewers value practical understanding over memorized definitions. Connect every answer to a real-world use case.